

MECANIQA AUTOMOTIVE TECH – ARACAJU

Autor: João Victor Carvalho de Oliveira
Kaio Santos de Oliveira
Marcelo Henrique Ribeiro Silva
Marco Antonio Almeida Silva
Nivton Amaral Alves
Robert de Araújo Costa

Vitória da Conquista, 2026

Agenda

O objetivo dessa apresentação é apresentar a evolução da API MecaniQA, destacando as novas entidades, padrões de projeto, DTOs, Mappers e o Diagrama de Atividade.

1. Introdução:

Contextualização e evolução da MecaniQA API

2. Objetivos:

Principais entregas da Sprint

3. Requisitos:

Novas entidades e regras de negócio

4. Tecnologias:

Java, Spring Boot e recursos utilizados

5. Modelagem OO:

Classes, Controllers, Repositories, DTOs, Mappers e Builder

Introdução



- Continuação do desenvolvimento da Web API da oficina mecânica;
- Expansão da aplicação para processos mais complexos do negócio;
- Introdução de padrões de projeto e DTOs;
- Manutenção da persistência dos dados em memória.

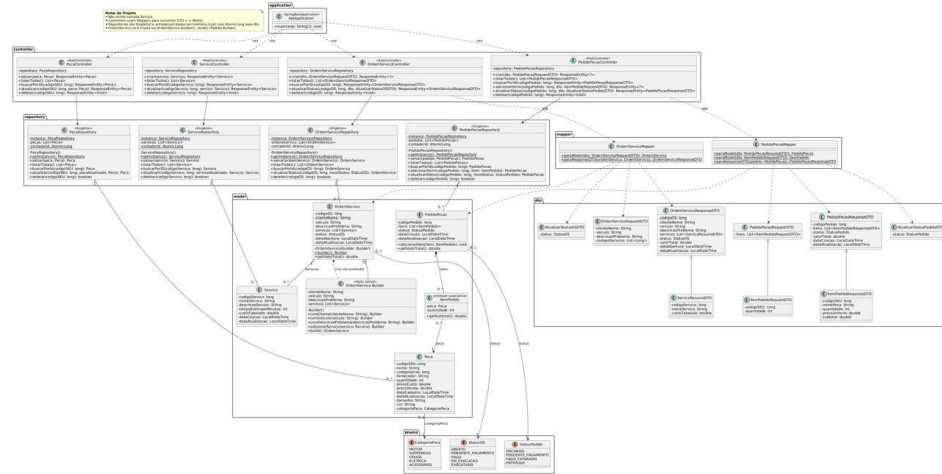
Objetivos

- Objetivos do projeto:
 - Criar as entidades **Ordem de Serviço e Pedido de Peças;**
 - Criar a **entidade associativa entre Peças e Pedidos;**
 - Implementar o **Padrão de Projeto Builder;**
 - Criar **DTOs e Mappers;**
 - Desenvolver um **Diagrama de Atividade de um endpoint.**



Requisitos

- Modelagem do nosso processo:
 - **Ordem de Serviço (OS)**
Representa o atendimento e os serviços realizados pela oficina;
 - **Pedido de Peças**
Representa a solicitação e o acompanhamento das peças necessárias;
 - **Entidade Associativa**
Relaciona as peças aos pedidos, permitindo múltiplas peças e suas respectivas quantidades.



Tecnologias

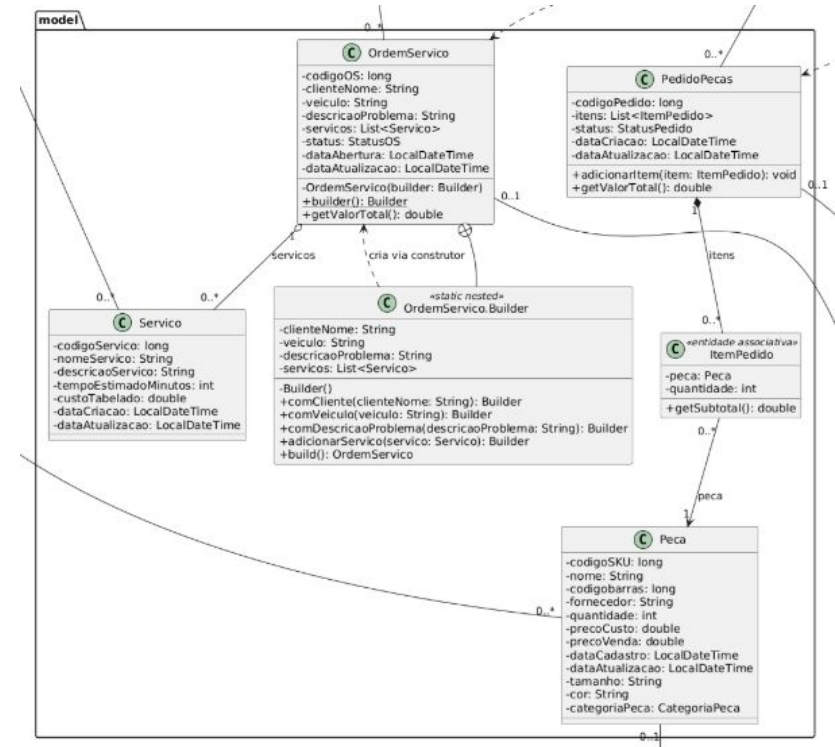
- Tecnologias e recursos utilizados:
 - **Java**
Linguagem utilizada para o desenvolvimento do backend;
 - **Spring Boot**
Framework utilizado na construção da Web API;
 - **Padrão Builder**
Utilizado para a construção da Ordem de Serviço;
 - **DTOs e Mappers**
Utilizados para separar os dados trafegados pela API das entidades de domínio.



Modelagem OO

- Estrutura de aplicação:

- **Models**
Representam as entidades e regras do domínio, como OrdemServico, PedidoPecas e Peca;
- **Controllers**
Responsáveis pelo recebimento e processamento das requisições da API;
- **Repositories**
Responsáveis pelo armazenamento dos dados em memória;
- **DTOs e Mappers**
Realizam a separação e conversão entre os dados da API e os Models;
- **Builder**
Responsável pela construção estruturada da Ordem de Serviço.



Modelagem OO

OrdemServico.java

```
102 public static class Builder {
103
104     private String clienteNome;
105     private String veiculo;
106     private String descricaoProblema;
107     private List<Servico> servicos = new ArrayList<>();
108
109     private Builder() {
110         // privado: só se cria um Builder através de OrdemServico.builder()
111     }
112
113     public Builder comCliente(String clienteNome) {
114         this.clienteNome = clienteNome;
115         return this;
116     }
117
118     public Builder comVeiculo(String veiculo) {
119         this.veiculo = veiculo;
120         return this;
121     }
122
123     public Builder comDescricaoProblema(String descricaoProblema) {
124         this.descricaoProblema = descricaoProblema;
125         return this;
126     }
127 }
```

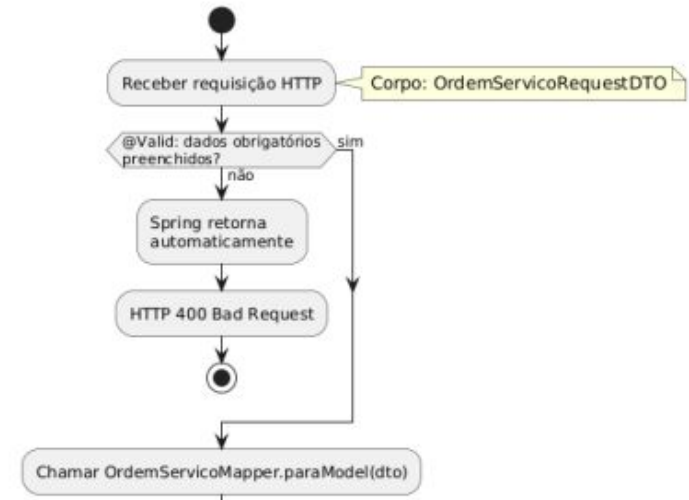
```
126     }
127
128     public Builder adicionarServico(Servico servico) {
129         this.servicos.add(servico);
130         return this;
131     }
132
133     public OrdemServico build() {
134         return new OrdemServico(this);
135     }
136 }
137
138 }
```

Modelagem OO

- Fluxo lógico do Controller:
 - Recebimento da requisição HTTP;
 - Validação dos dados;
 - Conversão do DTO para Model;
 - Construção da Ordem de Serviço com Builder;
 - Salvamento no Repository;
 - Conversão para Response DTO;
 - Retorno da resposta.

POST /api/ordens-servico

Diagrama de Atividade — POST /api/ordens-servico (US01)
(visão lógica do OrdemServicoController)

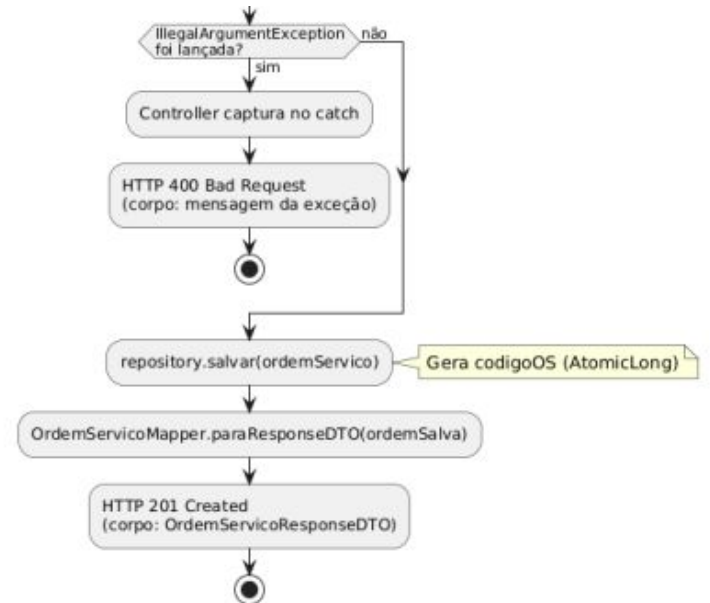
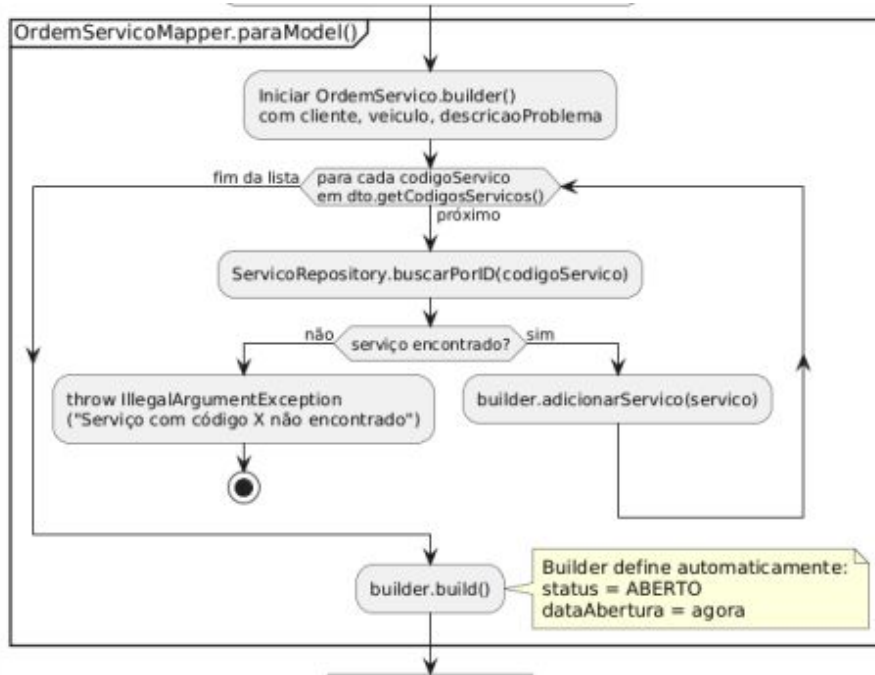


Modelagem OO

OrdemServicoController.java

```
17 @RestController
18 @RequestMapping("/api/ordens-servico")
19 public class OrdemServicoController {
20
21     private final OrdemServicoRepository repository = OrdemServicoRepository.getInstance();
22
23     // ===== CREATE (US01) =====
24     @PostMapping
25     public ResponseEntity<?> criar(@Valid @RequestBody OrdemServicoRequestDTO dto) {
26         OrdemServico ordemServico;
27         try {
28             ordemServico = OrdemServicoMapper.paraModel(dto);
29         } catch (IllegalArgumentException e) {
30             return ResponseEntity.badRequest().body(e.getMessage());
31         }
32
33         OrdemServico ordemSalva = repository.salvar(ordemServico);
34         return ResponseEntity.status(HttpStatus.CREATED)
35             .body(OrdemServicoMapper.paraResponseDTO(ordemSalva));
36     }
37 }
```

Modelagem OO



Modelagem OO

OrdemServiceMapper.java

```
22 public static OrdemServico paraModel(OrdemServicoRequestDTO dto) {
23     OrdemServico.Builder builder = OrdemServico.builder()
24         .comCliente(dto.getClienteNome())
25         .comVeiculo(dto.getVeiculo())
26         .comDescricaoProblema(dto.getDescricaoProblema());
27
28     ServicoRepository servicoRepository = ServicoRepository.getInstance();
29
30     for (Long codigoServico : dto.getCodigosServicos()) {
31         Servico servico = servicoRepository.buscarPorID(codigoServico);
32         if (servico == null) {
33             throw new IllegalArgumentException(
34                 "Serviço com código " + codigoServico + " não encontrado");
35         }
36         builder.adicionarServico(servico);
37     }
38
39     return builder.build();
40 }
```

Considerações finais



Principais resultados:

- Criação das entidades **Ordem de Serviço e Pedido de Peças;**
- Implementação da **Entidade Associativa** entre Peças e Pedidos;
- Implementação do **Padrão Builder;**
- Utilização de **DTOs e Mappers;**
- Desenvolvimento do **Diagrama de Atividade.**

O projeto amplia a base da API MecaniQA, permitindo representar novos processos da oficina e aplicar padrões de projeto e boas práticas na organização do código.

Referências



MECANIQA AUTOMOTIVE TECH.

OAT 2 — Construindo a Web API Fundacional
da MecâniQA

Prorgama de Trainee 2026.2.

GitHub. mecaniQA-api-aracaju. Repositório
do projeto da equipe.

[github.com/JottavDEV/mecani
QA-api-aracaju](https://github.com/JottavDEV/mecaniQA-api-aracaju)